

Лабораторная работа №4

Управляющие структуры

Александр Эдуардович Аскеров

2025-10-25

Докладчик

- Аскеров Александр Эдуардович
- Кафедра теории вероятностей и кибербезопасности
- Российский университет дружбы народов им. П. Лумумбы

Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

Задание

1. Используя Jupyter Lab, повторите примеры из раздела 4.2.
2. Выполните задания для самостоятельной работы.

Поэлементные операции над многомерными массивами

Повторим примеры из раздела поэлементные операции над многомерными массивами.

Для матрицы 4×3 рассмотрим поэлементные операции сложения и произведения её элементов:

```
a = rand(1:20, (4, 3))
```

```
4×3 Matrix{Int64}:
```

```
16 14 20  
19 8 2  
11 9 1  
15 13 20
```

```
sum(a)
```

```
148
```

```
sum(a, dims = 1)
```

```
1×3 Matrix{Int64}:
```

```
61 44 43
```

```
sum(a, dims = 2)
```

```
4×1 Matrix{Int64}:
```

```
50  
29  
21  
48
```

Рисунок 1: Поэлементные операции над многомерными массивами 1

Поэлементные операции над многомерными массивами

```
prod(a)
525837312000

prod(a, dims = 1)
1×3 Matrix{Int64}:
 50160 13104 800

prod(a, dims = 2)
4×1 Matrix{Int64}:
 4480
 304
 99
 3900
```

Рисунок 2: Поэлементные операции над многомерными массивами 2

Поэлементные операции над многомерными массивами

Для работы со средними значениями можно воспользоваться возможностями пакета Statistics:

```
import Pkg
Pkg.add("Statistics")
using Statistics

Updating registry at '~/.julia/registries/General.toml'
Resolving package versions...
Updating ~/.julia/environments/v1.11/Project.toml
[10745b16] + Statistics v1.11.1
No Changes to ~/.julia/environments/v1.11/Manifest.toml

mean(a)
12.333333333333334

mean(a, dims = 1)
1×3 Matrix{Float64}:
 15.25  11.0  10.75

mean(a, dims = 2)
4×1 Matrix{Float64}:
 16.666666666666668
  9.666666666666666
  7.0
 16.0
```

Рисунок 3: Поэлементные операции над многомерными массивами 3

Повторим примеры из раздела транспонирование, след, ранг, определитель и инверсия матрицы.

Для выполнения таких операций над матрицами, как транспонирование, диагонализация, определение следа, ранга, определителя матрицы и т.п. можно воспользоваться библиотекой (пакетом) `LinearAlgebra`:

Транспонирование, след, ранг, определитель и инверсия матрицы

```
import Pkg
Pkg.add("LinearAlgebra")
using LinearAlgebra

Resolving package versions...
Updating `~/julia/environments/v1.11/Project.toml`
[37e2e46d] + LinearAlgebra v1.11.0
No Changes to `~/julia/environments/v1.11/Manifest.toml`

b = rand(1:20, (4, 4))

4x4 Matrix{Int64}:
16 15 13 10
14 16 10 7
13 17 9 14
13 16 5 15

transpose(b)

4x4 transpose{::Matrix{Int64}} with eltype Int64:
16 14 13 13
15 16 17 16
13 10 9 5
10 7 14 15
```

Рисунок 4: Транспонирование, след, ранг, определитель и инверсия матрицы 1

Транспонирование, след, ранг, определитель и инверсия матрицы

```
tr(b)
56

diag(b)
4-element Vector{Int64}:
 16
 16
  9
 15

rank(b)
4

inv(b)
4x4 Matrix{Float64}:
 0.173508  0.00985222 -0.484488  0.257252
-0.227696  0.192118  0.168835 -0.0946987
 0.0738916 -0.0935961  0.231527 -0.221675
 0.0678788 -0.182266  0.0941434  0.0186897

det(b)
1827.0

pinv(b)
4x4 Matrix{Float64}:
 0.173508  0.00985222 -0.484488  0.257252
-0.227696  0.192118  0.168835 -0.0946987
 0.0738916 -0.0935961  0.231527 -0.221675
 0.0678788 -0.182266  0.0941434  0.0186897
```

Рисунок 5: Транспонирование, след, ранг, определитель и инверсия матрицы 2

Вычисление нормы векторов и матриц, повороты, вращения

Повторим примеры из раздела вычисление нормы векторов и матриц, повороты, вращения.

Для вычисления нормы используется `LinearAlgebra.norm(x)`.

Евклидова норма:

$$\|\vec{X}\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

p-норма:

$$\|\vec{A}\|_p = \left(\sum_{i=1}^n |a_i|^p \right)^{1/p}$$

Вычисление нормы векторов и матриц, повороты, вращения

```
X = [2, 4, -5]  
norm(X)
```

```
6.708203932499369
```

```
p = 1  
norm(X, p)
```

```
11.0
```

Рисунок 6: Вычисление нормы векторов и матриц, повороты, вращения 1

Вычисление нормы векторов и матриц, повороты, вращения

Евклидово расстояние между двумя векторами $\vec{X}$ и $\vec{Y}$ определяется как $\|\vec{X} - \vec{Y}\|_2$.

```
X = [2, 4, -5]
```

```
Y = [1, -1, 3]
```

```
norm(X - Y)
```

```
9.486832980505138
```

```
sqrt(sum((X - Y).^2))
```

```
9.486832980505138
```

Рисунок 7: Вычисление нормы векторов и матриц, повороты, вращения 2

Угол между двумя векторами $\vec{X}$ и $\vec{Y}$ определяется как $\cos^{-1} \left(\frac{\vec{X}^T \vec{Y}}{\|\vec{X}\|_2 \|\vec{Y}\|_2} \right)$.

```
acos((transpose(X) * Y) / (norm(X) * norm(Y)))  
2.4404307889469252
```

Рисунок 8: Вычисление нормы векторов и матриц, повороты, вращения 3

Вычисление нормы векторов и матриц, повороты, вращения

Вычисление нормы для двумерной матрицы:

```
d = [5 -4 2; -1 2 3; -2 1 0]
```

```
3x3 Matrix(Int64):
```

```
5  -4  2  
-1  2  3  
-2  1  0
```

```
opnorm(d)
```

```
7.147682841795258
```

```
p = 1
```

```
opnorm(d, p)
```

```
8.0
```

```
rot180(d)
```

```
3x3 Matrix(Int64):
```

```
0  1 -2  
3  2 -1  
2 -4  5
```

```
reverse(d, dims = 1)
```

```
3x3 Matrix(Int64):
```

```
-2  1  0  
-1  2  3  
5  -4  2
```

```
reverse(d, dims = 2)
```

```
3x3 Matrix(Int64):
```

```
2  -4  5  
3  2  -1  
0  1  -2
```

Рисунок 9: Вычисление нормы векторов и матриц, повороты, вращения 4

Матричное умножение, единичная матрица, скалярное произведение

Повторим примеры из раздела матричное умножение, единичная матрица, скалярное произведение.

```
A = rand(1:10, (2, 3))  
2x3 Matrix{Int64}:  
 8 7 3  
 8 9 8  
  
B = rand(1:10, (3, 4))  
3x4 Matrix{Int64}:  
 7 6 4 9  
 8 4 2 8  
 9 4 6 7  
  
A*B  
2x4 Matrix{Int64}:  
139 88 64 149  
200 116 98 200  
  
Matrix{Int}(I, 3, 3)  
3x3 Matrix{Int64}:  
 1 0 0  
 0 1 0  
 0 0 1  
  
X = [2, 4, -5]  
Y = [1, -1, 3]  
dot(X, Y)  
-17  
  
X*Y  
-17
```

Рисунок 10: Матричное умножение, единичная матрица, скалярное произведение

Факторизация, специальные матричные структуры

Повторим примеры из раздела факторизация, специальные матричные структуры.
Решение систем линейных алгебраических уравнений $Ax = b$:

```
A = rand(3, 3)

3x3 Matrix{Float64}:
 0.568652  0.500527  0.360144
 0.534665  0.677178  0.873399
 0.954807  0.723092  0.38112

x = fill(1.0, 3)

3-element Vector{Float64}:
 1.0
 1.0
 1.0

b = A*x

3-element Vector{Float64}:
 1.4293226471103924
 2.085242465382307
 2.0590189432101584

A\b

3-element Vector{Float64}:
 1.0000000000000104
 0.9999999999999825
 1.0000000000000078
```

Рисунок 11: Факторизация, специальные матричные структуры 1

Julia позволяет вычислять LU-факторизацию и определяет составной тип факторизации для его хранения:

```
Alu = lu(A)
LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0   0.0   0.0
 0.559972  1.0   0.0
 0.595567  0.256649  1.0
U factor:
3×3 Matrix{Float64}:
 0.954807  0.723092  0.38112
 0.0       0.272267  0.659983
 0.0       0.0      -0.0362223
```

Рисунок 12: Факторизация, специальные матричные структуры 2

Факторизация, специальные матричные структуры

Различные части факторизации могут быть извлечены путём доступа к их специальным свойствам:

```
Alu.P
3x3 Matrix(Float64):
 0.0  0.0  1.0
 0.0  1.0  0.0
 1.0  0.0  0.0

Alu.p
3-element Vector{Int64}:
 3
 2
 1

Alu.L
3x3 Matrix(Float64):
 1.0      0.0      0.0
 0.559972  1.0      0.0
 0.595567  0.256649  1.0

Alu.U
3x3 Matrix(Float64):
 0.954807  0.723092  0.38112
 0.0       0.272267  0.659983
 0.0       0.0       -0.0362223
```

Рисунок 13: Факторизация, специальные матричные структуры 3

Факторизация, специальные матричные структуры

Исходная система уравнений $Ax = b$ может быть решена или с использованием исходной матрицы, или с использованием объекта факторизации:

```
A\b
3-element Vector{Float64}:
 1.00000000000000104
 0.9999999999999825
 1.0000000000000078

A\b
3-element Vector{Float64}:
 1.00000000000000104
 0.9999999999999825
 1.0000000000000078
```

Рисунок 14: Факторизация, специальные матричные структуры 4

Аналогично можно найти детерминант матрицы:

```
det(A)
0.009416468305519874

det(A|u)
0.009416468305519874
```

Рисунок 15: Факторизация, специальные матричные структуры 5

Julia позволяет вычислять QR-факторизацию и определяет составной тип факторизации для его хранения:

```
Aqr = qr(A)
LinearAlgebra.QRCompactWV{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3x3 LinearAlgebra.QRCompactWQ{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3x3 Matrix{Float64}:
-1.23324  -1.08422  -0.839793
 0.0      -0.237339 -0.573758
 0.0      0.0      0.8321416
```

Рисунок 16: Факторизация, специальные матричные структуры 6

По аналогии с LU-факторизацией различные части QR-факторизации могут быть извлечены путём доступа к их специальным свойствам:

```
Aqr.Q
3x3 LinearAlgebra.QRCompactWYQ(Float64, Matrix{Float64}, Matrix{Float64})

Aqr.R
3x3 Matrix{Float64}:
-1.23324 -1.08422 -0.039793
 0.0      -0.237559 -0.575788
 0.0      0.0      0.0321416

Aqr.Q'*Aqr.Q
3x3 Matrix{Float64}:
 1.0 -4.33681e-19 -1.11022e-16
-5.55112e-17 1.0 5.55112e-17
 0.0 0.0 1.0
```

Рисунок 17: Факторизация, специальные матричные структуры 7

Факторизация, специальные матричные структуры

Примеры собственной декомпозиции матрицы A :

```
Asym = A + A'  
  
3x3 Matrix(Float64):  
 1.1373  1.03519  1.31495  
 1.03519  1.35436  1.59649  
 1.31495  1.59649  0.76224  
  
AsymEig = eigen(Asym)  
  
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}  
values:  
3-element Vector{Float64}:  
 -0.6842426906280551  
  0.21050904259272207  
  3.7276337263061743  
vectors:  
3x3 Matrix{Float64}:  
 -0.318328  0.780627  -0.537855  
 -0.479003  -0.622064  -0.619349  
  0.818061  -0.0604781  -0.571943  
  
AsymEig.values  
  
3-element Vector{Float64}:  
 -0.6842426906280551  
  0.21050904259272207  
  3.7276337263061743
```

Рисунок 18: Факторизация, специальные матричные структуры 8

Факторизация, специальные матричные структуры

```
AsymEig.vectors  
3x3 Matrix{Float64}:  
-0.318328  0.780627  -0.537855  
-0.479003  -0.622064  -0.619349  
 0.818061  -0.0604781 -0.571943  
  
inv(AsymEig) * Asym  
3x3 Matrix{Float64}:  
 1.0      1.44329e-15  5.27356e-16  
2.22045e-16  1.0      5.55112e-16  
6.66134e-16  4.44089e-16  1.0
```

Рисунок 19: Факторизация, специальные матричные структуры 9

Факторизация, специальные матричные структуры

Далее рассмотрим примеры работы с матрицами большой размерности и специальной структуры.

```
n = 1000
A = randn(n, n)
Asym = A + A'
issymmetric(Asym)

true
```

Рисунок 20: Факторизация, специальные матричные структуры 10

Пример добавления шума в симметричную матрицу (матрица уже не будет симметричной):

```
Asym_noisy = copy(Asym)
Asym_noisy[1, 2] += 5eps()
issymmetric(Asym_noisy)

false
```

Рисунок 21: Факторизация, специальные матричные структуры 11

Факторизация, специальные матричные структуры

В Julia можно объявить структуру матрицы явно, например, используя Diagonal, Triangular, Symmetric, Hermitian, Tridiagonal и SymTridiagonal:

```
Asym_explicit = Symmetric(Asym_noisy)
1000×1000 Symmetric{Float64, Matrix{Float64}}:
 1.97979  4.92095  2.72556  - 0.8148918 -1.07828  0.267106
 4.92095  1.91722  0.14943  -1.47488 -1.14523  1.32846
 2.72556  0.14943  1.14485  0.42953  2.50673  1.62347
-2.19548  0.953689 -0.504952  3.68985 -0.362123  1.96302
 0.921507 -0.272272 -2.81856 -3.26458  1.17483  1.755
-1.37832  1.86345  0.123664  0.280946  2.46559 -0.0245747
 0.811966 -0.608376  0.42439 -1.72456  2.52536  0.530222
-0.788945  2.32134  0.856031  1.54762  1.63086 -0.774984
 0.456613  1.357  1.85343  0.430141  0.88457  3.11337
-1.01917 -0.989902 -3.06728  0.794361 -1.35462  0.517828
 0.493996 -2.85916  1.47934 -0.514341 -2.26154 -1.61878
-2.53897 -1.90444  1.00483  0.923304  8.61564 -0.548393
-3.86267  1.73026 -0.321148 -1.71041 -0.237281 -1.03682
⋮
-0.77639 -0.170836  0.754215  2.82506 -0.691589 -1.74761
 0.9156 -0.917288 -1.20891  0.881184 -0.606206 -2.48406
 0.91361  1.96638  2.9694 -1.61657  1.63507 -2.79375
 1.15491  2.24778 -1.16357  1.95763  1.55434  0.548914
 1.48713 -0.388219 -2.08784  0.398908 -1.86181  1.62296
 0.983123  0.757819  2.78696 -0.337588  0.0979149  0.0107647
-0.178611  0.148795 -1.24067 -1.79122 -2.11442 -1.38865
-0.446136 -2.82627 -1.71341 -1.17799 -4.03639  0.746943
 0.153011  0.381763 -0.358054 -0.725583 -0.9292  2.98071
 0.0148918 -1.47488  0.42953  1.71729  1.98977 -2.8436
-1.07828 -1.14523  2.50673  1.98977  2.08893  1.44913
 0.267106  1.32846  1.62347 -2.8436  1.44913  0.695979
```

Рисунок 22: Факторизация, специальные матричные структуры 12

Факторизация, специальные матричные структуры

Далее для оценки эффективности выполнения операций над матрицами большой размерности и специальной структуры воспользуемся пакетом BenchmarkTools:

```
import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools

Resolving package versions...
Installed Compat ~ v4.16.1
Installed BenchmarkTools ~ v1.6.3
Updating ~/julia/environments/v1.11/Project.toml
[6e4b88f9] + BenchmarkTools v1.6.3
Updating ~/julia/environments/v1.11/Manifest.toml
[6e4b88f9] + BenchmarkTools v1.6.3
[346a2180] + Compat v4.16.1
[9abbd945] + Profile v1.11.0
Precompiling project...
 974.3 ms ✓ Compat
 813.5 ms ✓ Compat ~ CompatLinearAlgebraExt
1598.8 ms ✓ BenchmarkTools
3 dependencies successfully precompiled in 4 seconds. 49 already precompiled.

@time eigvals(Asym);
54.112 ms (21 allocations: 7.99 MiB)

@time eigvals(Asym_noisy);
591.147 ms (27 allocations: 7.93 MiB)

@time eigvals(Asym_explicit);
53.588 ms (23 allocations: 7.99 MiB)
```

Рисунок 23: Факторизация, специальные матричные структуры 13

Факторизация, специальные матричные структуры

Далее рассмотрим примеры работы с разреженными матрицами большой размерности.

Использование типов `Tridiagonal` и `SymTridiagonal` для хранения трёхдиагональных матриц позволяет работать с потенциально очень большими трёхдиагональными матрицами:

```
n = 1000000;  
A = SymTridiagonal(randn(n), randn(n - 1))  
@btime eigmax(A)  
  
573.027 ms (44 allocations: 183.11 MiB)  
6.177465995098636
```

Рисунок 24: Факторизация, специальные матричные структуры 14

При попытке задать подобную матрицу обычным способом и посчитать её собственные значения, вы скорее всего получите ошибку переполнения памяти:

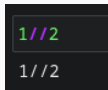
```
B = Matrix(A)  
OutOfMemoryError()
```

Рисунок 25: Факторизация, специальные матричные структуры 15

Повторим примеры из раздела общая линейная алгебра.

Julia также поддерживает общую линейную алгебру, что позволяет, например, работать с матрицами и векторами рациональных чисел.

Для задания рационального числа используется двойная косая черта:

A screenshot of a code editor showing two lines of Julia code. The first line is `1//2` with the first '1' in green and the second '1' in purple. The second line is `1//2` in white text on a dark background.

```
1//2  
1//2
```

Рисунок 26: Общая линейная алгебра 1

В следующем примере показано, как можно решить систему линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой (для избежания проблемы с переполнением используем BigInt):

```
A = Rational * Matrix(Rational(BigInt))(rand(1:10, 3, 3))/10
3x3 Matrix(Rational(BigInt)):
2//5  3//10  3//10
3//10  1      3//5
1//5  1//5  7//10

x = fill{1, 3}
3-element Vector{Int64}:
 1
 1
 1

b = A * Rational * x
3-element Vector{Rational{BigInt}}:
 1
19//10
11//10

A * Rational * b
3-element Vector{Rational{BigInt}}:
 1
 1
 1

[u(A * Rational)]
10(Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64})
1 factor:
3x3 Matrix(Rational{BigInt}):
 1  0  0
3//4  1  0
1//2  2//31  1
0 factor:
3x3 Matrix(Rational{BigInt}):
2//5  3//10  3//10
0  31//40  3//5
0  0  163//310
```

Рисунок 27: Общая линейная алгебра 2

Произведение векторов

1. Задайте вектор v . Умножьте вектор v скалярно сам на себя и сохраните результат в `dot_v`.

```
v = [9, 8, 7, 6]
dot_v = dot(v, v)
230
```

Рисунок 28: Задание 1

2. Умножьте v матрично на себя (внешнее произведение), присвоив результат переменной `outer_v`.

```
outer_v = v * v'
4x4 Matrix{Int64}:
 81  72  63  54
 72  64  56  48
 63  56  49  42
 54  48  42  36
```

Рисунок 29: Задание 2

1. Решить СЛАУ с двумя неизвестными.

```
A = [1 1; 1 -1]
b = [2; 3]
A \ b
```

```
2-element Vector{Float64}:
 2.5
-0.5
```

```
A = [1 1; 2 2]
b = [1; 2]
if (det(A) != 0)
    A \ b
else
    print("No solution")
end
```

```
No solution
```

```
A = [1 1; 2 2]
b = [2; 5]
if (det(A) != 0)
    A \ b
else
    print("No solution")
end
```

```
No solution
```

Рисунок 30: Задания a, b, c

```
A = [1 1; 2 2; 3 3]
b = [1; 2; 3]
A \ b

2-element Vector{Float64}:
 0.4999999999999999
 0.5

A = [1 1; 2 1; 1 -1]
b = [2; 1; 3]
A \ b

2-element Vector{Float64}:
 1.5000000000000004
-0.9999999999999997

A = [1 1; 2 1; 3 2]
b = [2; 1; 3]
A \ b

2-element Vector{Float64}:
-0.9999999999999989
 2.9999999999999982
```

Рисунок 31: Задания d, e, f

2. Решить СЛАУ с тремя неизвестными.

```
A = [1 1 1; 1 -1 -2]
b = [2; 3]
A \ b

3-element Vector{Float64}:
 2.2142857142857144
 0.35714285714285704
-0.5714285714285712
```

```
A = [1 1 1; 2 2 -3; 3 1 1]
b = [2; 4; 1]
A \ b

3-element Vector{Float64}:
-0.5
 2.5
 0.0
```

```
A = [1 1 1; 1 1 2; 2 2 3]
b = [1; 0; 1]
if (det(A) != 0)
    A \ b
else
    print("No solution")
end

No solution
```

```
A = [1 1 1; 1 1 2; 2 2 3]
b = [1; 0; 0]
if (det(A) != 0)
    A \ b
else
    print("No solution")
end

No solution
```

Рисунок 32: Задания а, b, c, d

1. Приведите приведённые ниже матрицы к диагональному виду.

```
A = [1 -2; -2 1]
Aeig = eigen(A)
result = diagm(Aeig.values)

2×2 Matrix{Float64}:
-1.0  0.0
 0.0  3.0

B = [1 -2; -2 3]
Beig = eigen(B)
result = diagm(Beig.values)

2×2 Matrix{Float64}:
-0.236068  0.0
 0.0       4.23607

C = [1 -2 0; -2 1 2; 0 2 0]
Ceig = eigen(C)
result = diagm(Ceig.values)

3×3 Matrix{Float64}:
-2.14134  0.0  0.0
 0.0      0.515138  0.0
 0.0      0.0  3.6262
```

Рисунок 33: Задание 1

2. Вычислите.

```
A = [1 -2; -2 1]
A^10

2x2 Matrix(Int64):
 29525 -29524
-29524 29525

B = [5 -2; -2 5]
sqrt(B)

2x2 Matrix(Float64):
 2.1889 -0.45685
-0.45685 2.1889

C = [1 -2; -2 1]
C^(1/3)

2x2 Symmetric{ComplexF64, Matrix{ComplexF64}}:
 0.971125+0.433013im -0.471125+0.433013im
-0.471125+0.433013im 0.971125+0.433013im

D = [1 2; 2 3]
sqrt(D)

2x2 Matrix{ComplexF64}:
 0.568864+0.351578im 0.920442-0.217287im
 0.920442-0.217287im 1.48931+0.134291im
```

Рисунок 34: Задание 2

3. Найдите собственные значения матрицы A .

```
A = [140 97 74 168 131; 97 106 89 131 36; 74 89 152 144 71; 168 131 144 54 142; 131 36 71 142 36]
Aeig = eigen(A)
Aeig.values

5-element Vector{Float64}:
-128.49322764802145
-55.887784553057
 42.752167279318854
 87.16111477514488
542.467730146614
```

Рисунок 35: Собственные значения матрицы A

Создайте диагональную матрицу из собственных значений матрицы A .

```
Adiag = diagm(Aeig.values)
5x5 Matrix{Float64}:
-128.493  0.0  0.0  0.0  0.0
 0.0 -55.8878  0.0  0.0  0.0
 0.0  0.0  42.7522  0.0  0.0
 0.0  0.0  0.0  87.1611  0.0
 0.0  0.0  0.0  0.0  542.468
```

Рисунок 36: Диагональная матрица из собственных значений матрицы A

Создайте нижнедиагональную матрицу из матрицы A .

```
LowerTriangular(A)
5x5 LowerTriangular{Int64, Matrix{Int64}}:
140  .  .  .  .
 97 106  .  .  .
 74  89 152  .  .
168 131 144  54  .
131  36  71 142 36
```

Рисунок 37: Нижнедиагональная матрица из матрицы A

Оцените эффективность выполняемых операций.

```
@btime diagn(Aeig.values)
59.606 ns (2 allocations: 272 bytes)
5×5 Matrix{Float64}:
-128.493  0.0  0.0  0.0  0.0
 0.0 -55.8878  0.0  0.0  0.0
 0.0  0.0  42.7522  0.0  0.0
 0.0  0.0  0.0  87.1611  0.0
 0.0  0.0  0.0  0.0  542.468

@btime LowerTriangular(A)
178.259 ns (1 allocation: 16 bytes)
5×5 LowerTriangular{Int64, Matrix{Int64}}:
140  -  -  -  -
 97 106  -  -  -
 74  89 152  -  -
168 131 144  54  -
131  36  71 142  36
```

Рисунок 38: Оценка эффективности выполняемых операций

Линейная модель экономики может быть записана как СЛАУ

$$x - Ax = y,$$

где элементы матрицы A и столбца y – неотрицательные числа. По своему смыслу в экономике элементы матрицы A и столбцов x, y не могут быть отрицательными числами.

1. Матрица A называется продуктивной, если решение x системы при любой неотрицательной правой части y имеет только неотрицательные элементы x_i . Используя это определение, проверьте, являются ли матрицы продуктивными.

```
function productive_matrix(matrix, size)
    ans = true
    E = [1 0; 0 1]
    Y = rand(0:1000, size)
    S = E - matrix
    X = S\Y
    for i in 1:size
        for j in 1:size
            if X[i] < 0
                ans = false
                break
            else
                ans = true
            end
        end
    end
    return ans
end

A = [1 2; 3 4]
productive_matrix(A, 2)

false

B = 1/2 * A
productive_matrix(B, 2)

false

C = 1/10 * A
productive_matrix(C, 2)

true
```

Рисунок 39: Задание 1

2. Критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все элементы матрицы

$$(E - A)^{-1}$$

являются неотрицательными числами. Используя этот критерий, проверьте, являются ли матрицы продуктивными.

```
function productive_matrix_2(matrix, size)
    ans = true
    E = [1 0; 0 1]
    new_matrix = E - matrix
    inv_new_matrix = inv(new_matrix)
    for i in 1:1:size
        for j in 1:1:size
            if inv_new_matrix[i, j] < 0
                ans = false
                break
            else
                ans = true
            end
        end
    end
    return ans
end

A = [1 2; 3 1]
productive_matrix_2(A, 2)

false

B = 1/2 * A
productive_matrix_2(B, 2)

false

C = 1/10 * A
productive_matrix_2(C, 2)

true
```

Рисунок 40: Задание 2

3. Спектральный критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все её собственные значения по модулю меньше 1. Используя этот критерий, проверьте, являются ли матрицы продуктивными.

```
function productive_matrix_3(matrix, size)
    ans = true
    eigenvalues = eigvals(matrix)
    for i in 1:size
        if abs(eigenvalues[i]) >= 1
            ans = false
            break
        else
            ans = true
        end
    end
    return ans
end

A = [1 2; 3 1]
productive_matrix_3(A, 2)
false

B = 1/2 * A
productive_matrix_3(B, 2)
false

C = 1/10 * A
productive_matrix_3(C, 2)
true

D = [0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]
productive_matrix_3(D, 2)
true
```

Рисунок 41: Задание 3

Изучены возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.